

■ 레드 블랙 트리

▷ 개념

- 노드의 색은 블랙, 레드로 나눈다.
노드의 Left, Right 노드가 공집합이므로,
공란 노드는 공란 노드로 보며,
공란 노드의 색은 블랙으로 본다

- 큰 규칙은 2개다

1) 루트 노드부터 공란 노드 까지의
모든 블랙 노드의 수는,
모든 공란 노드의 루트들에서 같다

2) 레드 노드와 레드 노드는
소문 (부모 - 자식 관계)
일 수 없다

(+ 루트 노드는 항상 블랙이며, 삽입 노드는 레드이다)
→ 위 규칙은 목적으로 하며,
노드의 삽입, 삭제, 재배치를 한다

▷ 기본 함수

○ Left Rotate

FUNCTION leftRotate ('부모')

'오른쪽' = '부모'의 right

'부모'의 right = '오른쪽'의 left

'오른쪽'의 부모 = '부모'의 부모

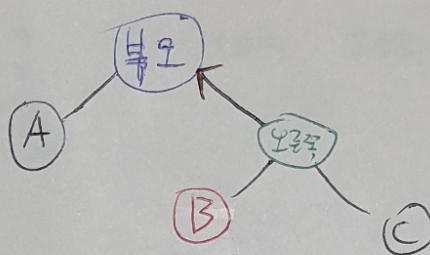
'오른쪽'의 left = '부모'

'부모'의 부모 = '오른쪽'

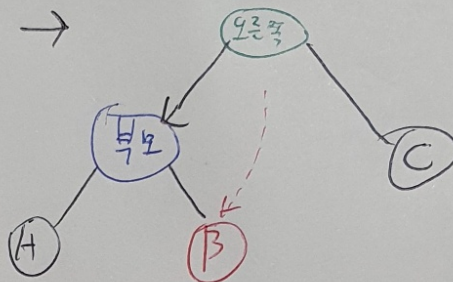
ENDFUNCTION

→ 왼쪽 (반시계)으로 회전하는데,
기존 부모라리에 right가 오고,
기존 left라리에 부모가 온다
기존 right의 left는,
기존 부모의 right로 연결한다

Ex)



→

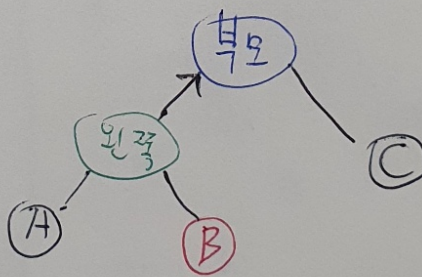


○ Right Rotate

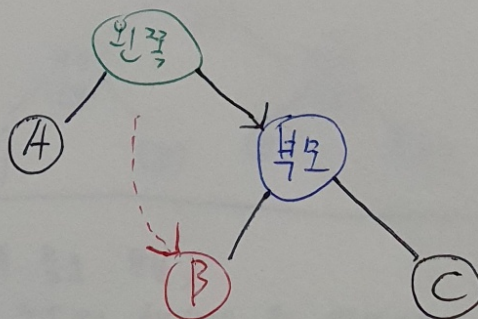
→ Left Rotate의 반대 방향인,
시계 방향으로 회전이다.

기존 부모라리에 left가 오고,
기존 right라리에 부모가 온다
기존 left의 right는,
기존 부모의 left로 연결한다

Ex)



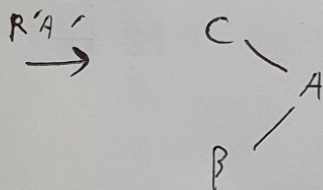
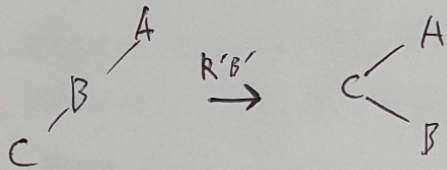
→



* LL, LR, RL, RR 은

앞선 L, R 함수의 조합을 의미한다

Ex)



D 삽입 함수 구현

- 삽입 함수는, 경계 조건 처리하는
기존 트리의 삽입과 동일하다.
- 삽입 완료 후, 삽입된 노드는 기존으로
경계 함수를 호출한다 (삽입노드의 기본색은 레드이다)

FUNCTION insertFixup ('기준 노드')

IF '기준 노드' == '루트'

'기준 노드'의 색 = 블랙

RETURN

ENDIF

WHILE '기준 노드'의 부모가 레드

IF '삼촌 노드'가 블랙

(※ 삼촌 노드는 부모 노드의 형제 노드)

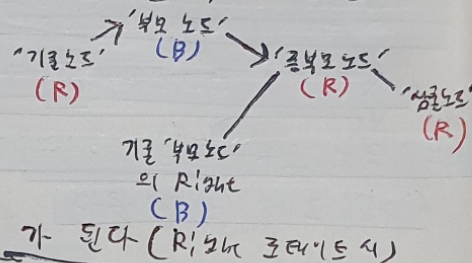
'증조부 노드'를 기준으로,

'부모 노드'가 증조부 위로 이동, L 또는 R 회전

기준 증조부 위치의 '부모 노드'의
색상은 블랙으로 전환

기준 삼촌 노드 위치의 '증조부 노드'의
색상은 레드로 전환

* '기준 노드'의 색상은 기본으로
레드 이므로, 이 결과로



BREAK (반복 종료)

ELSE ('삼촌 노드'도 '부모노드'와
마찬가지로 레드일시)

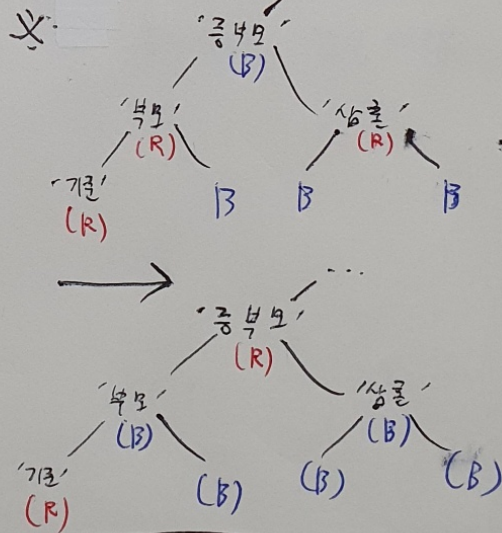
'부모 노드'와 '삼촌 노드'의 색은
블랙으로 바뀐다.

'증조부 노드'의 색은 레드로 바뀐다.

'기준 노드' = '증조부 노드'

GOTO WHILE

(색이 레드로 바뀐 증조부 노드는
기준으로 하며, while 문은
반복한다.)



(함수 종료 부분)

루트 노드의 색상이 무리 반복문들은 종료
조건으로 변했다면, 다시 경계를 바꾼다

→ 위 의사코드 내용은 정리하면,

- 삽입시 부모 노드가 블랙이면,
아무것도 처리 않는다
- 부모노드가 레드면, 1톤 사이가
레드-레드 이므로, 규칙에 위반 되기에,
정리를 한다

1) 삽입이 블랙이면,
부모 노드가 증부모 위로,
증부모 노드가 기존 삽입 위로
회전한다. (+ 부모는 블랙, 증부모는 레드로 전환)

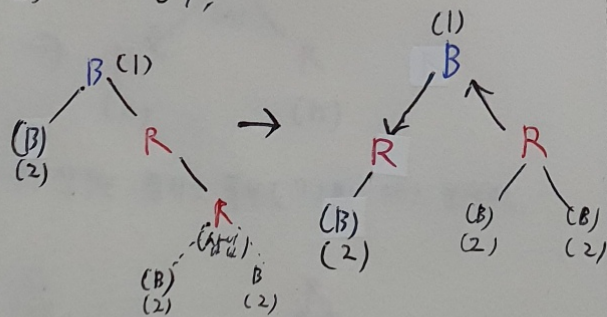
2) 삽입도 레드면,
부모, 삽입이 블랙이 되고,
증부모가 레드가 된다.
증부모는 기존 노드로 되어 증부모
반복한다.

(마리 위로 올라가려는 쪽에서
증부모 쪽으로 전이되는 것까
같다)

0 삽입 함수가 어떻게 두 규칙을 따는지..

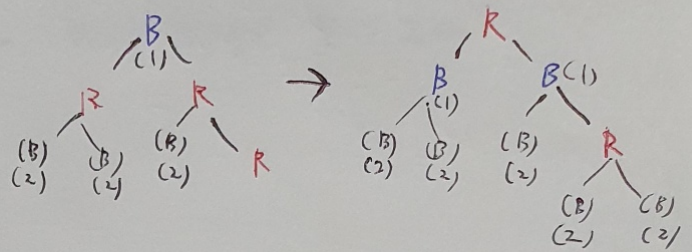
- 삽입 함수는 두 규칙을 위반한다.

1) 의 경우,



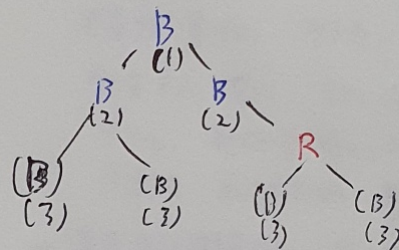
→ 이러한, 극도로 빠른 증분가치로 모든 극도로 빠른
분할의 속은 (2)로 유지된다

2) 의 경우



3) 비한가리로 (2)로 유지된다.

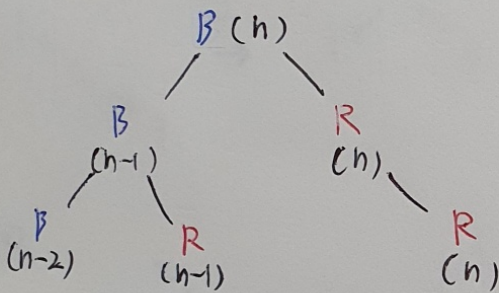
* 만약 이 증분가치, 극도로 빠른
그렇다면 함수의 중요 특별에 의해
증분가치 노드를 다시 블랙이 된다.



이러한 극도로 빠른, 상용 각방에 의해
(극도로 빠른의 각방들이 모두 레드였던 경우)
일시적으로 레드가 됐던 게네스가,
유일하게 모든 극도의 분할의 속가
1씩 증가하는 경우이다.

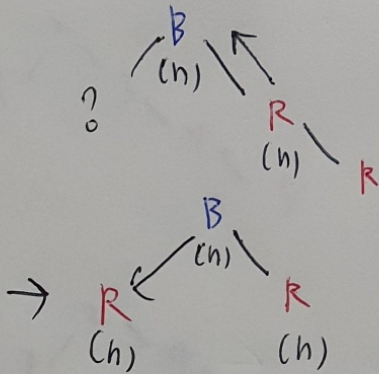
○ 좌측은 동관 일반화

- 같은 성질을 통해, 3대에서의 삽입 과정에
중복부에서 중단 노트까지의 모든 노트에서의
분할이 수가 동일함을 보였다.
- 이를 이용해 특정 노트의 좌측은
해당 노트에서 중단 노트까지의 분할이
수로 정의하라. 그렇다면,

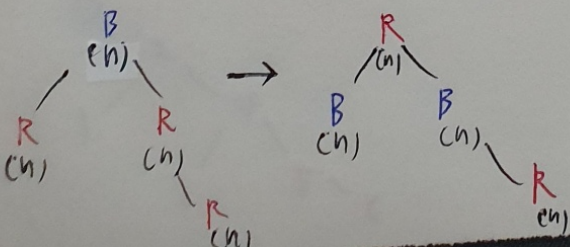


→ 이러한 좌측의 좌측이 n 일시,
현재노드가 분할이된 현재의 좌측은 $n-1$,
현재가 레드면 현재의 좌측은 n
임을 알 수 있다.

* 위 성질은 로테이션 상충의 예로
보이면,



* 위 성질은 중복부 포함(?)은 예로 보이면,

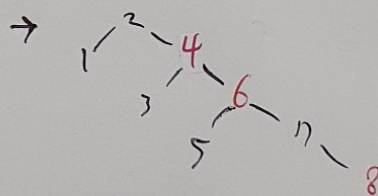
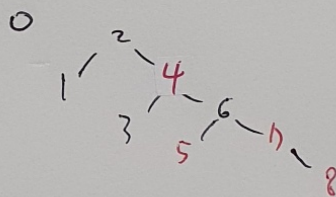
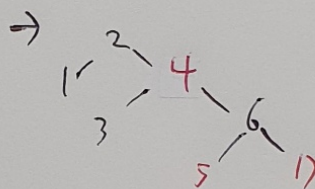
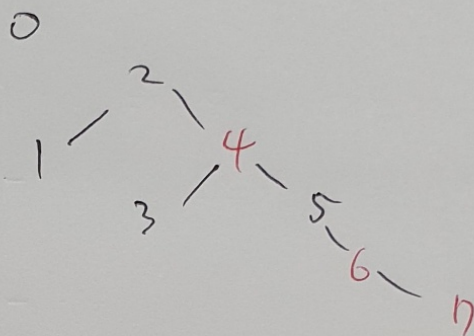
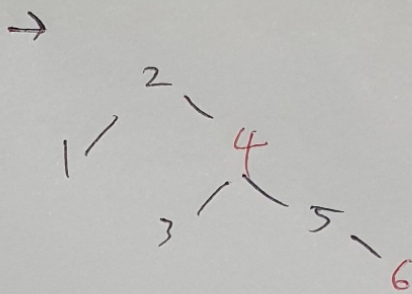
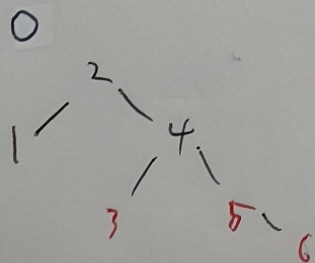
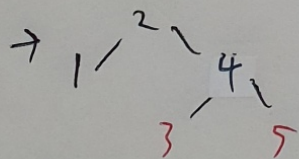
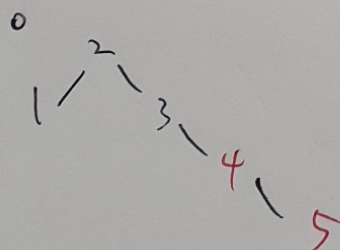
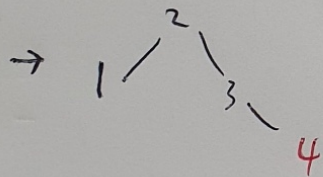
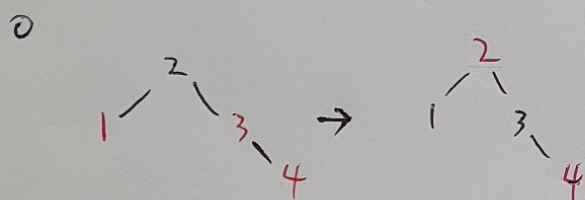
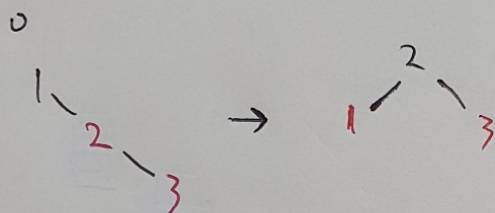
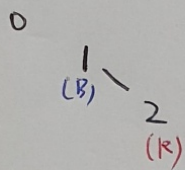
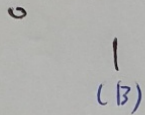


→ 이렇게 할시 3대에서 보였던 내용이,
좌측은 동관 일반화 노트에서도
동일이 적용됨을 알 수 있다

○ 레드 분할 트리의 편향 분배 완화 효과

- 삽입된 노트는 모두 레드이기때,
레드는 상승 분리가 기대된다.
- 반면 상승 효과는 레드가 분할이 되는
경우는 없다.
(중복부 포함이 경우도, 미려한 위가 레드가된다)
상승 효과 없이 상승 노드가 분할이 되는
경우는, 반대쪽 행렬에서 빌어 올려
주기가 훨씬 타당 분할을 내려오는 것이다.
- 극도로 기울어진 한 쪽으로 편향되어
삽입된 경우, 삽입의 타당 분배에서는
리스크인 '중복부 포함'이,
극도 인근에서는 로테이션 상충은
중대,
편향된 노트들이 복귀한 노트 공간을
이득함은 알 수 있다.

D Tree 3/8 16/16 5/14



→ 이러한 '증분보존'과 '회전상승'은 동태
적증분수에서, 부족한 보충은
노드들이 재조정된다

D 사례 함수 구현

0 erase 함수

- 기존 트리의 사례 함수와 비례는 같으며, 분기 번호 'fix 노드' 선정과 대체 노드의 생성 변경 코드가 복잡함

FUNCTION erase ('사례 키')

트리의 검색으로, '사례 키'에 대응되는 노드를 찾는다

노드가 없다면, RETURN

노드가 있다면, 그 노드를 '사례 노드'

① IF '사례 노드'의 Left 이 없다면,

'사례 노드'의 Right 를
사례 노드 위치로 대체한다

'fix 노드' = 기존 '사례 노드'의 Right
(현 '사례 노드' 위치 노드)

② ELSE IF '사례 노드'의 Right 이 없다면

'사례 노드'의 Left 를
사례 노드의 위치로 대체한다

'fix 노드' = 기존 '사례 노드'의 Left

ELSE (양쪽 자식이 모두 있다면)

'사례 노드'의 Right 의 자식 중, 가장
왼쪽에 위치한 자식은

'인계 노드'로 지정한다

(※ Right 의 왼쪽 자식이 없다면,
Right 가 인계 노드가 된다)

③ IF Right == '인계 노드' 라면,

Right 가 '사례 노드'를 대체한다

(Right 는 왼쪽 자식이 없으므로, 문제 없음)

'fix 노드'의 Right 의 Right
(Right 이 오른쪽 자식)

④ ELSE (Right ≠ '인계 노드')

(Right 가 왼쪽 자식이 있다면)

Right 가
가장 왼쪽
자식

'인계 노드'가 '사례 노드'를 대체한다

'인계 노드'의 Right 가, '인계 노드'를
대체한다

'fix 노드' = '인계 노드'의 Right

(현 기존 '인계 노드'의
위치에 있는 노드)

ENDIF

'인계 노드'의 생성은 기존 '사례 노드'의
생성으로 바꾼다

(즉, 이 부분은 ③, ④ 예만 해당된다)

ENDIF

인계 처리가 끝난 '사례 노드'를 삭제한다

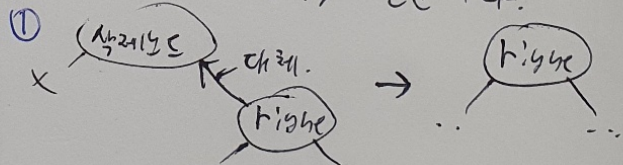
IF ('fix 노드'가 유효하는 공간에,
기존에 존재했던 노드의 생성이
정리되었다면)

CALL eraseFix up ('fix 노드')
(fix 노드를 기준으로 재정리)

ENDIF

ENDFUNCTION

→ ①, ② 의 경우, 단순하다.



3 Right 가 사례 노드를 대체한다.

이 때 Right 는 'fix 노드'로,

Right 가 유효하는 공간에 있던 노드는

'사례 노드'의 생성이 정리되었다면,

레드 블랙 트리의 규칙 2개 중

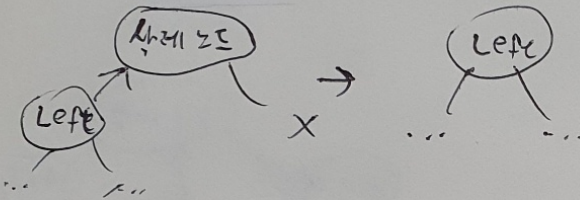
'모든 노드의 자식들의 볼륨도 같다'가
복합적으로 깨지고,

'레드나 레드 블랙 노드인 것'도
조건부로 깨질 수 있다.

따라서 'fix 노드'의 Right 는 기존에 eraseFix up으로

반대로 '삭제노드'가 레드였다면,
 픽스업 노드의 색상이 수정이 될 것이므로 위쪽쪽은
 위치를 바꿀 필요가 없다.
 따라서 평형은 꼭 지켜진다.

(2)



①과 조각이 똑같다.

'픽스 노드' = Left.

삭제 노드가 평형이었다면,

'픽스노드'를 기준으로 재평형

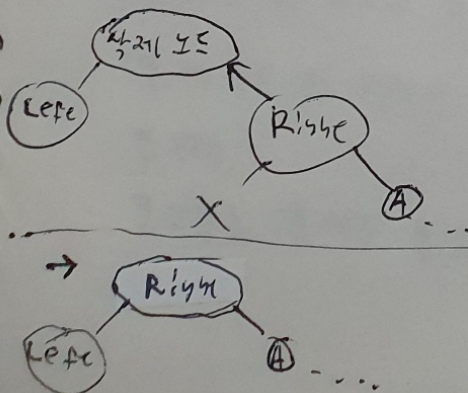
(3) (4)

- 3, 4의 경우, 복잡해진다.

공통적 = 3, '삭제노드'를 인계하는 '인계노드'
 가 '픽스업 노드'가 아니라,
 '인계노드'의 Right가 '픽스업 노드'
 가 된다.

(3)

- '삭제노드'의 Right가, 왼쪽 자식이
 없어, Right 자식이 '인계노드'가
 되는 경우다.



- 이 때, 앞선 경우에서와 같이,

(3) (4)의 경우 '인계노드'가 '삭제노드'의
 색상을 인계한다

따라서 '삭제노드'가 위치하고 현 Right가
 있는 노드는 규칙들을 지켜라 한다

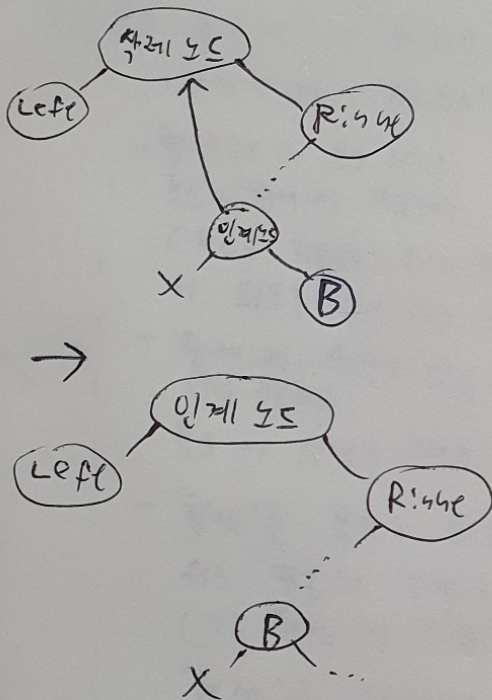
- 하지만 'Right'의 Right인 (A)는,
 'Right'의 색상은 인계하지 않아도,
 현 (A)가 위치하는 곳은 문제가 있을 수 있다.

- 만약 (A)가 위치하는 곳에 원래 존재하던
 노드 ('Right')의 색상이 평형이었다면,
 문제가 발생하지만,
 노드 (A) ('Right'의 Right)를 기준으로
 재평형

* 노드 'Right'의 색상이 레드였다면,
 ①, ②와 마찬가지로 문제가 없음

④

- 가장 복잡한 경우다.
- '삭제 노드'를 먼저 지우는 '인계 노드'는 'Right'의 가장 왼쪽 자손이다.
- '최소값 노드'는 '인계 노드'의 Right



- ③과 마찬가지로, '인계 노드'는 '삭제 노드'의 색상은 먼저 지우기에, '인계 노드'가 위치하게 되는 곳은 문제가 발생하지 않는다.

- 하지만 '인계 노드'를 먼저 지우는 '인계 노드'의 Right인 (B)는, '인계 노드'의 색상을 먼저 지우기에, 조건부 문제 발생한다.

- 마찬가지로 '인계 노드'의 색상을 지우기 전에, 우선적으로 ③ 위치에 문제가 발생하지 않게, ③은 지우는 것을 피한다.

* '인계 노드'의 색상을 지우기 전에, ①-②-③과 마찬가지로 문제가 발생

2 eraseFixup 함수

- erase 후, 루트 (과거 위치 노드-점)으로 재설정 목적으로 호출되는 함수이다

FUNCTION eraseFixup ('fix 노드')

WHILE ('fix 노드'가 루트가 아니며, 불일)

① IF 'fix 노드'가 레드일 때
(※ 'fix 노드' = 'fix 노드' 부모의 다른 자식)

- 'fix 노드'가 부모 위치로, 부모가 'fix'의 자식 노드, ('fix'는 기존 노드 부모의 자식)

이 되도록 부모 기존 위치로 정렬

- 'fix'의 색상은 검정으로 전환, (기존 부모 위치)

부모의 색상은 레드로 전환

- 'fix'를 부모가 회전 전환으로 색을 교환으로 인제한, ('fix'의 자식)으로

'fix'를 재설정

ENDIF

(※ ① 이후로 'fix'는 반드시 검정)

② IF 'fix'의 자식들이 모두 검정

'fix'의 색상은 레드로 전환

'fix'는 'fix'의 부모로 재설정의

GOTO WHILE (루트 반복)

ELSE ('fix'의 자식 중 루트 노드 이 레드)

③ IF ('fix' 기존, 'fix'의 자식 중 가까운 자식이 레드, 먼 자식은 불일)

- '가까운 fix 자식'이 'fix'에 위치하고, 'fix'는 '가까운 fix 자식'의 자식이 되어 회전.

- '가까운 fix 자식'은 검정, 'fix'는 레드로 전환

ENDIF

④

(fix 기존 'fix'에 위치 'fix'에 레드)

(※ '가까운 fix 자식'의 색은 레드)

- 'fix'가 부모의 위치로, 부모가 'fix'의 자식이 되도록 회전 ('fix'는 여전히 'fix'의 자식)

- 'fix'의 색상은 기존 부모의 색으로, 부모의 색은 검정으로, 'fix'의 자식'은 검정으로 전환

- ④ < 1, 재설정되는 문제가 해결된다) RETURN

ENDIF

ENDWHILE

⑤ IF 'fix 노드'가 루트 노드 (null ptr)가 아니면

'fix 노드'의 색상은 검정으로 전환

ENDIF

ENDFUNCTION



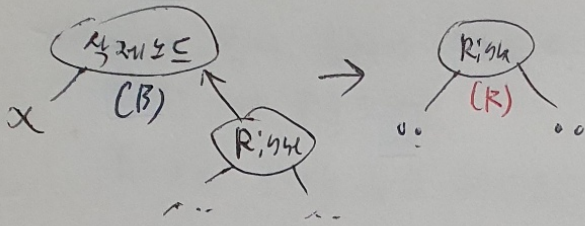
복잡하다. 하나씩 풀이 설명한다.

⑥ WHILE 문이 생략 (fix 노드나 레드)되는 경우

- 트리가 평형하다. 이 함수는 기존 노드가 평형일 때에만, 문제가 생기면 호출된다.

- 이 때에 fix 노드가 레드라면, fix 노드를 검정으로 바꾸는 것 (⑤)만으로도 문제가 쉽게 해결된다.

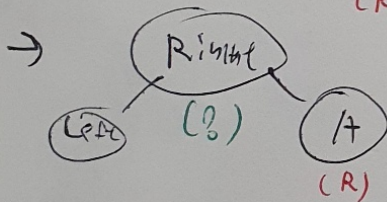
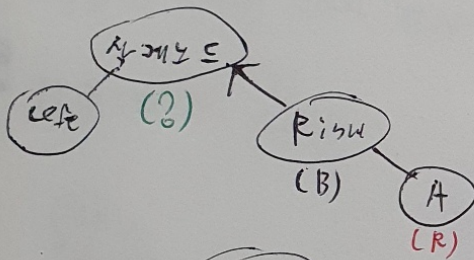
- 말선 ① 을 예로 보자.



→ B였던 자리에 R가 왔기에 문제가 되는 거다.

R은 B로 바꾸는 것만으로 문제는 해결 된다

- 말선 ③ 을 예로 보자



→ ③ ④의 경우 '현재 노드'는 '삭제노드'의 자식인 관계라기엔, 현재노드는 문제가 없다.

하지만 이동 후 (A) 위치를 보면

2개가 문제가 될 수 있다.

1) 고정적인 (A)의 라수가 1 줄어든다.
(R'이 B가 되었으므로)

2) (?)가 R일 경우,
R-R가 1줄어든다

위 두 문제는, (A)를 점령으로 바꾸는 것만으로도 문제는 해결 된다.

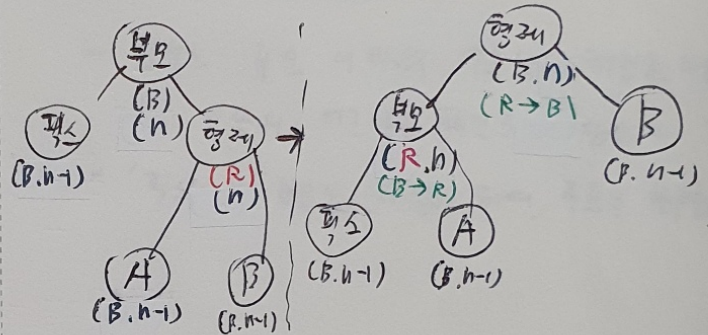
(※ 반대로 픽스가 점령인 경우, 문제가 발생한다.)
WHILE문의 반복으로 처리한다.

○ ① 이 포착되는 경우.

- ①은 문제를 직접 해결하는 게 아니나,
문제가 해결 되도록 환경들을 조성해준다

- ②, ③, ④는 '현재'가 점령일 때만
권제로 한다.

- ①은 '현재'가 레드일 때에 조건부
드레인으로, 각 노드별 라수의 변동 없이,
'현재'가 점령이 되도록 점령한다.



→ 우선 점령 권을 보자. 현재가 레드나일 건,
부모, 현재의 자식들이 모두 점령은 뜯힌다.
따라서 기존 삭제노드의 라수가 h이었다 하면,
위처럼 라수가 확장된다.

→ 점령 권을 보자. 이 메시에서는 '부모' L회전
하고, 부모를 레드, 현재를 점령으로 전환한다.
'위치'의 높이는 바뀌지만, '위치'의 노드의 라수는 변하지 않는다.

한편 픽스의 현재는 고정적으로
점령인 (A)로 재점령 된다

→ 이러한 ①은 '현재'가 점령이 되도록
점령 라수, ②, ③, ④에서 처리할 수 있게
한다.

○ ②가 종료되는 경우

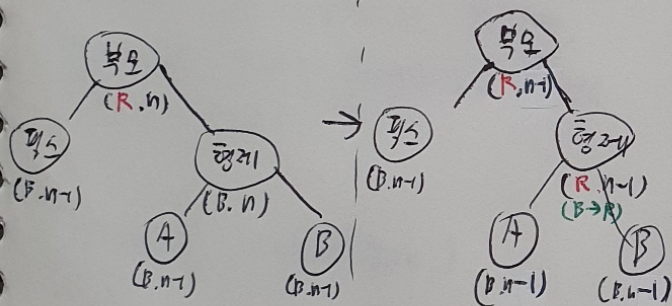
- ②는 형제, 형제의 자식들이 모두 정렬된 때 종료되며, 조건부로 문제를 즉시 해결하도록 한다.

(내용은 간단하다.)

- 형제는 레드로 바꾼다.
- 부모는 'fixs'로 리셋되기에 즉각적으로 반부한다.

부모의 세상에 따라 결과가 달라진다.

1) 부모가 R 인 경우



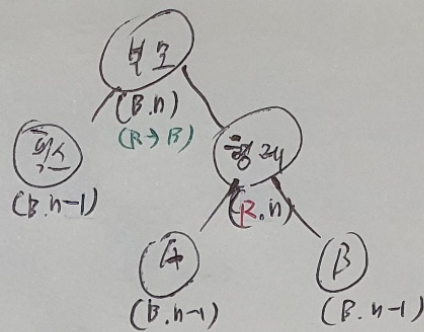
- 정렬 편을 보라. fixs는 n-1, 형제는 n의 어긋난 상황이다. (특히 같은 정렬인데도)

- 정렬 편을 보라. fixs, 형제의 자식들이 노드가 해결됐다.

(다만 형제는 날려서, 부모 기준으로 라줄 간의 노드가 해결된 거라, 그 위에는 여전히 자식이 복원되)

- 이 때, WHILE 루프가 돌고, 'fixs' (여기서 부모)가 레드이므로, WHILE을 빠져 나와, 'fixs'를 정렬으로 전환한다.

→



- 정렬으로 전환시, 자식이 어긋났던 것이 부모 이하, 부모 이상에서 모두 해결된다

2) 부모가 B 인 경우

- 정렬은 부모 이하의 어긋남은 해결될 것 같, 부모 이상의 어긋남은 해결되지 않았다.

- 'fixs'는 부모로 리셋되기에, 즉각적으로 반부한다

부모는 'fixs'로 리셋되기에

○ ③ 이 트로피되는 경우

- ③ 은 ① 이 거래여 '형제'가 블랙
이고,

② 가 복령되어 '형제'의 자식 중
최소 소이 레드인 상황에서,

'형제'의 자식 중 최소 복령
면 노드가 블랙인 때 트로피된다.

즉, '형제'는 '형제'는 블랙,

'형제' 자식'은 블랙,

'가까운 형제 자식'은 레드이다

- ③ 의 트로피는 ① 과 비슷하다.

문제는 꼭 짝 재배치하는 양이지만,

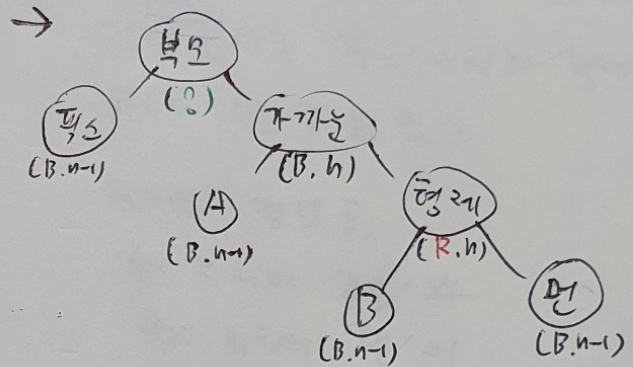
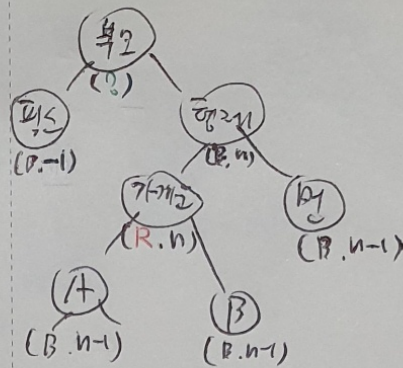
④ 에서 문제는 재배치하도록 형제는
안 돌아온다

(내용은 아래와 같다.)

- '가까운 형제 자식'이 '형제'의 자식,
'형제'가 '가까운 형제 자식'의 자식이
되도록, '형제'를 트로피한다

- '가까운 형제 자식'은 검은 노드,
'형제'는 레드노드 전환한다.

- '형제'는 '가까운 형제 자식'으로
재배치한다



→ 이런제, 기존 '형제'의 노드와 자식
변동 없이,

④ 에서 트로피한 후 바로,

'형제' 자식'이 레드가 되도록
정렬한다.

○ ④ 가 호출되는 경우

- ④ 는 ① 은 거리 2, ② 가 복귀되며,
③ 은 거리 3 상태가 되며,

'형제'는 검정, '먼 형제'의 라식'은 레드
가 확장된다.

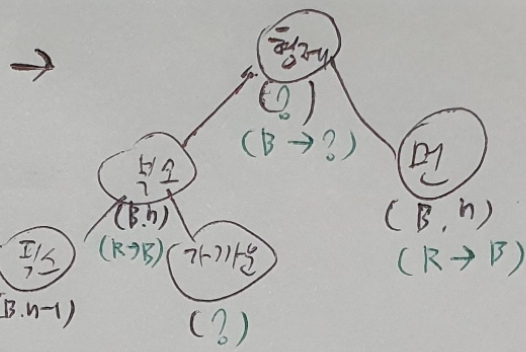
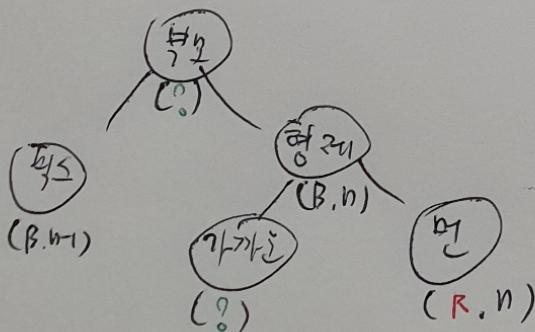
'가까운 형제'의 라식'의 색은 모른다
(③이 커리콜하면 검정된 테리만)

- ④ 는 확장각으로 불려온 즉시
해결한다

(나뭇잎은 아개와 같다)

- '형제'가 '부모'가 되도록,
'부모'가 '형제'의 라식'이 되도록,
'부모' 기록 희면 한다.

- '부모'를 검정으로,
'형제'는 '부모'의 기록 색은 검정,
'먼 형제'의 라식'은 레드로
색을 전환한다



→ 재귀 연산, '우리'를 기준으로 부가.
정렬 연산은 '팩스' 우리와 형제
라식'이 같아 나 있는 것이다.

하지만 정렬 연산,
'팩스' 우리의 '부모'와
'형제' 우리의 '먼'의
라식'이 같아 나 있는 것이다.

기록 부모가 R 이었다면,
'형제' (우리 '부모')의 라식'은 n,
기록 부모가 B 였다면,
'형제'의 라식'은 n+1 이 된다.
(이는 식제 연의 '부모'의 라식'과
일치한다)

→ '형제' (기록 우리 '부모') 이라,
이 상태에서 라식'이 같아 나 있는
문제가 모두 해결된다.
라식'을 즉시 종료해 주 된다